ME 144L Pre Lab: Introduction to DSCLab and Using Python

<u>Finding Signals and Statistics Using Python and Numpy</u>

Through developing the read and write processing code, the biggest issue I came across was that the csv file was not found correctly, leading to errors. After some fast troubleshooting, I determined that this issue was a result of the code being saved into my 'users' folder, rather than my 'downloads' folder, where the exported acceleration data was located. After this fix, the code worked perfectly.

Data Source	**Mean**	**Standard Deviation**
PhyPhox Sensor Database	9.814 [m/s^2]	0.0170 [m/s^2]
Raw data from Phone	9.803 [m/s^2]	0.0066 [m/s^2]
% Difference	0.11%	88.14%

Figure I: iPhone 14 statistics for PhyPhox Sensor Database vs. Real Recorded Variances

Analyzing the data from PhyPhox and the stationary phone test, the two means compare very nicely to one another. The standard deviation in the stationary phone is much lower than expected, likely as a result of a small sampling size (2.25-second period) and few external influences that could skew results.

Python Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import scipy.fftpack
from scipy import signal

plt.close('all') # closes any open plots
#-----------------------------------------------------------------
# Read comma-separated values (CSV) file
#-----------------------------------------------------------------
filename = 'raw_data_accelg.csv'
# note: edit the column names
csvdata = pd.read_csv(filename)
print('Data read...')

#-----------------------------------------------------------------
# Index into the columns using the renamed column header names
#-----------------------------------------------------------------
```

```
Tcsv = np.array(csvdata['Time (s)'])
accx = np.array(csvdata['Acceleration x (m/s^2)'])
accy = np.array(csvdata['Acceleration y (m/s^2)'])
accz = np.array(csvdata['Acceleration z (m/s^2)'])
acct = np.array(csvdata['Absolute acceleration (m/s^2)'])

# Standard Deviations & Mean

std_x = np.std(accx)
std_y = np.std(accy)
std_z = np.std(accz)
std_t = np.std(acct)

mean_x = np.mean(accx)
mean_y = np.mean(accy)
mean_z = np.mean(accz)
mean_t = np.mean(acct)

print(std_t)
print(mean_t)
```

Compute the Spectral Density for Acceleration Data

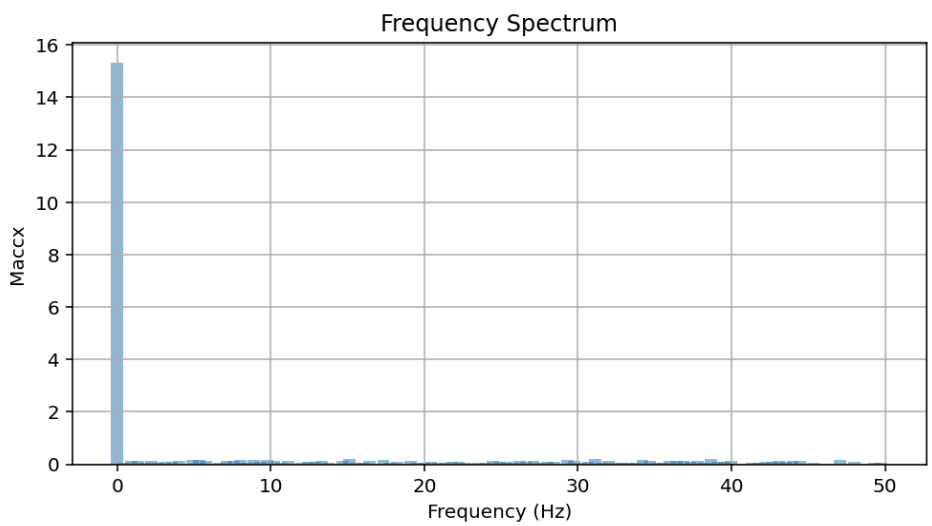

Figure II: Acceleration (x) Spectral plot

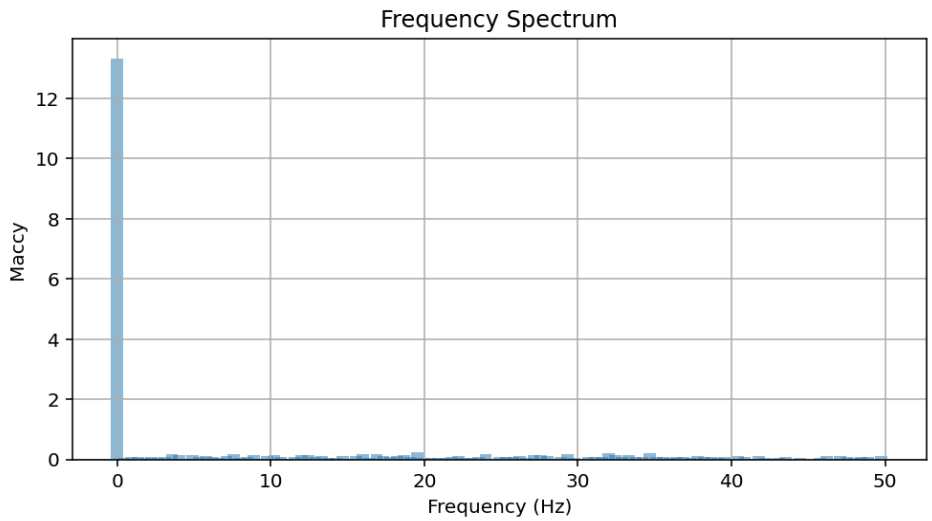

Figure III: Acceleration (y) Spectral plot

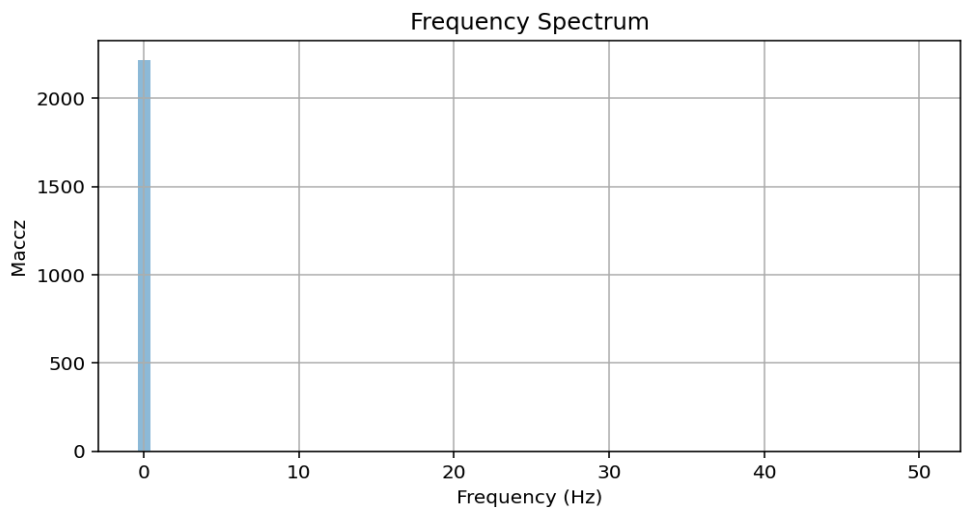

Figure IV: Acceleration (z) Spectral plot

RC Filter Model

The digital filter begins as a continuous transfer function representing an RC circuit low-pass filter, which describes the ratio of output to input in the frequency domain. To make this computable, the function is discretized, converting a continuous function into arrays for computational plotting, where variables are replaced by time delays. By algebraically rearranging this equation to solve for the current output, the current filtered value ‘xf[k]’ is calculated simply by summing weighted values of the current input, the previous input, and the previous filtered output. Below are the determined filters for x, y, z, and absolute data for an RC system of a 1 Ohm resistor and 2 Farad capacitor:

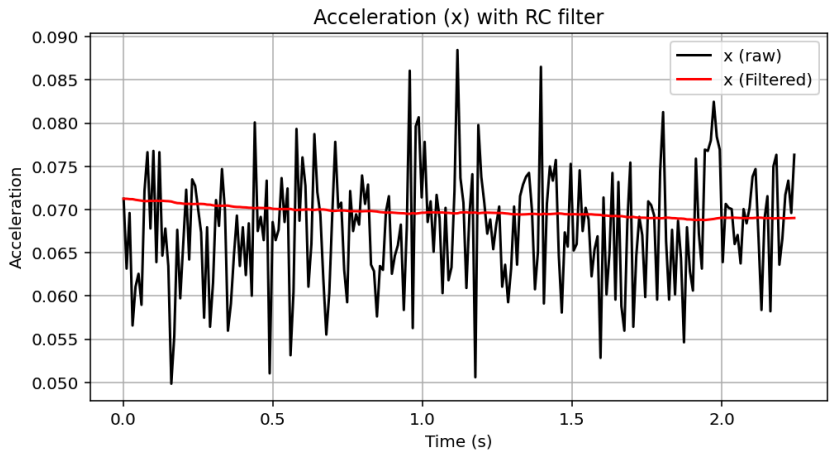

Figure V: Acceleration (x) with Digital RC Filter Applied

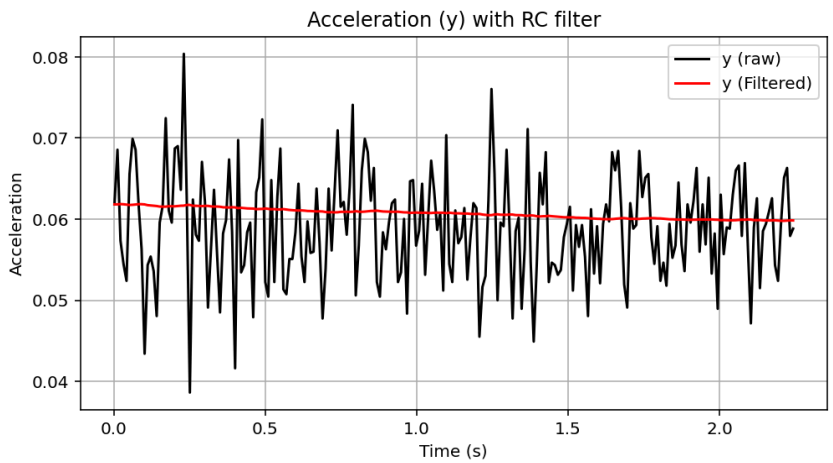

Figure VI: Acceleration (y) with Digital RC Filter Applied

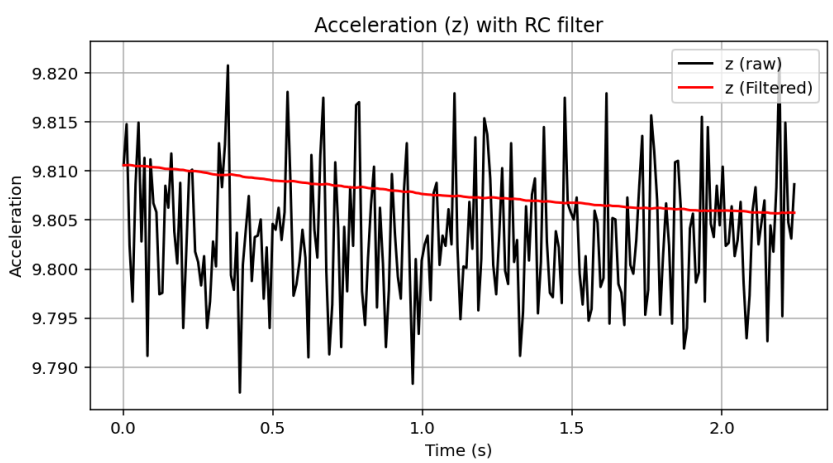

Figure VII: Acceleration (z) with Digital RC Filter Applied

Flywheel Model

The flywheel model describes the dynamic response of a rotating mass during a spin-down test, where the system decelerates from an initial speed due to friction. The friction is an energy loss that can be represented as a resistive torque, τ_B, and is directly proportional to the angular velocity of the system in the equation shown:

$$\tau_B = B\omega$$

The rotational inertia of the flywheel and damping coefficient B, act similarly to the RC circuit, and can be constructed to create a time constant, τ_t, unique to the flywheel setup. Below are the state equations for the flywheel to describe its angular acceleration and angular velocity.

$$\tau_t = \frac{J}{B}$$

$$\begin{bmatrix} \dot{\omega} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} -\frac{1}{\tau_t}\omega \\ \omega \end{bmatrix}$$